

Advance Database Systems
Lab Mid Project

Submitted by:

Muhammad Mahad Ahmad
FA23-BCS-122
BCS-A

Question 1 (Design Scenario — 45 marks)

Scenario:

You are designing the backend for an **e-commerce marketplace** like Amazon. The platform needs to handle:

- Products (name, description, category, price, brand, rating, stock)
- Users (name, email, location, purchase history)
- Orders (user, products, quantities, total cost, timestamp)
- Reviews (user, product, rating, review text, timestamp)
- Search functionality where customers can:
 - Do keyword search (e.g., “laptop”)
 - Similarity search (e.g., “hp leptop” → “HP Laptop”)
 - Hybrid search that ranks results by similarity + price relevance (e.g., closer to budget) + popularity (number of purchases).

Tasks:

1. Design the MongoDB schema with justification. Use embedded + referenced documents where appropriate.
2. Suggest indexes for optimizing searches (compound indexes on text + price).
3. Design APIs for:
 - /products/search?query=...
 - /users/<id>/orders
 - /products/<id>/reviews
 - /orders/<id>
4. Write one **aggregation pipeline** to find the “Top 5 most frequently purchased products in the last month, grouped by category.”

Task 1:

Using JSON style for the schema and following up with a "Design Justification" for each one.

1. users Collection:

Schema:

JSON:

```
{
  "_id": "ObjectId('...')",
  "name": "Mahad",
  "email": "mahad@cuilahore.edu.pk", // Unique Index
  "hashed_password": "bcrypt_hash_string...",
  "location": {
    "address": "123 Main St",
    "city": "Lahore",
    "country": "Pakistan"
  },
  "purchase_history": [
    "ObjectId('order_id_1')", // Reference to 'orders' collection
    "ObjectId('order_id_2')"
  ],
  "created_at": "ISODate('...')"
}
```

Design Justification:

- **Referencing (purchase_history):** A user's order history can grow indefinitely (an "unbounded array"). Embedding all orders would make the user document enormous, slow to load, and violate MongoDB's 16MB document size limit. Therefore, we store an array of ObjectIds referencing the orders collection. This is a "one-to-many" relationship where referencing is superior.
- **Embedding (location):** The user's location is "one-to-one" data. It's queried with the user record, not separately. Embedding it simplifies queries and avoids an unnecessary join.

2. products Collection

Schema:

JSON:

```
{
  "_id": "ObjectId(...)",
  "name": "HP Envy x360 14",
  "description": "Premium 2-in-1 laptop with 14-inch display.",
  "category": "Electronics",
  "brand": "HP",
  "price": 250,000,
  "stock": 150,
  "specs": { // Embedded sub-document
    "ram_gb": 16,
    "storage_ssd_gb": 1024,
    "processor": "Ryzen 7"
  },
  "review_summary": { // Embedded for fast reads
    "average_rating": 4.7,
    "review_count": 92
  },
  "purchase_count": 1204, // For "popularity" sorting
  "created_at": "ISODate(...)"
}
```

Design Justification:

- **Embedding (specs):** Product specifications are part of the core product data and are always displayed together. Embedding them is efficient.
- **Embedding (review_summary):** A product could have thousands of reviews. We must reference the actual reviews (in their own collection). However, we constantly need the average rating and count for display on product listing and search pages. We embed a summary object that is updated (e.g. by a trigger or backend logic) whenever a new review is added. This "Computed Pattern" saves a complex aggregation on every product load.
- **(purchase_count) Field:** This field is denormalized and incremented every time the product is part of a successful order. It directly supports the "popularity" ranking requirement for hybrid search.

3. reviews Collection

Schema:

JSON:

```
{
  "_id": "ObjectId(...)",
  "product_id": "ObjectId('product_id_1')", // Reference to 'products'
  "user_id": "ObjectId('user_id_1')",    // Reference to 'users'
  "rating": 5, // Number (1-5)
  "review_text": "Amazing laptop, battery life is fantastic!",
  "timestamp": "ISODate('...')"
```

Design Justification:

- **Separate Collection:** As mentioned, a product can have an unbounded number of reviews. A separate reviews collection is necessary.
- **Referencing:** We use product_id and user_id to link the review back to the relevant product and user. This allows us to query all reviews for a specific product (using an index on product_id) or all reviews by a specific user.

4. orders Collection

Schema:

JSON:

```
{
  "_id": "ObjectId(...)",
  "user_id": "ObjectId('user_id_1')", // Reference to 'users'
  "products": [
    {
      "product_id": "ObjectId('product_id_1')", // Reference for linking
      "name": "HP Envy x360 14",    // Snapshot
      "price_at_purchase": 250,000, // Snapshot
      "quantity": 1
    },
    {
```

```

    "product_id": "ObjectId('product_id_2')",
    "name": "Logitech MX Master 3",
    "price_at_purchase": 25,000,
    "quantity": 2
  }
],
"total_cost": 275,000,
"status": "shipped", // e.g., 'pending', 'paid', 'shipped', 'delivered'
"shipping_address": { // Snapshot
  "address": "123 Main St",
  "city": "Lahore",
  "country": "Pakistan"
},
"timestamp": "ISODate('...')"
}

```

Design Justification:

- **Embedding (products array):** An order has a finite, "one-to-few" relationship with its products. Embedding the product array is efficient.
- **Snapshotting:** This is the most critical part. We do not just reference the product _id. Why? Because the price or name in the products collection might change later. An order is a historical record of a transaction. We must "snapshot" the name and price_at_purchase inside the order document itself to maintain historical accuracy.
- **Snapshotting (shipping_address):** For the same reason, we embed the user's shipping address. The user might change their primary address in the users collection later, but this order must always be tied to the address it was shipped to.

Task 2: Indexing Strategy

Proposed Indexes:

1. On users collection:

1. `db.users.createIndex({ email: 1 }, { unique: true })`

- **Justification:** Ensures email addresses are unique and provides extremely fast lookups during user login or registration.

2. On products collection:

1. `db.products.createIndex({ name: "text", description: "text", brand: "text" })`

- **Justification:** This is the core Text Index. It supports the keyword search (e.g., "laptop") and similarity search (e.g., "leptop") for the `/products/search` API. It tokenizes and stems the content of these fields.

2. `db.products.createIndex({ category: 1, price: 1 })`

- **Justification:** A Compound Index to optimize browsing. It allows users to efficiently find all products in a "category," sorted by "price" (low to high).

3. `db.products.createIndex({ price: 1 })`

- **Justification:** A single-field index to support the "price relevance" (budget) part of the hybrid search.

4. `db.products.createIndex({ purchase_count: -1 })`

- **Justification:** A single-field index to support the "popularity" part of the hybrid search, allowing fast sorting by the most purchased items.

3. On reviews collection:

1. `db.reviews.createIndex({ product_id: 1, timestamp: -1 })`

- **Justification:** A Compound Index to power the `/products/<id>/reviews` API. It allows MongoDB to instantly find all reviews for a single `product_id` and return them pre-sorted by date (newest first).

4. On orders collection:

1. `db.orders.createIndex({ user_id: 1, timestamp: -1 })`

- **Justification:** A Compound Index to power the `/users/<id>/orders` API. It finds all orders for a `user_id` and returns them pre-sorted by date (most recent orders first), which is the most common query pattern.

Task 3: API Design

1. GET /products/search

- **Description:** Searches for products. Supports keyword/similarity search, filtering, and sorting for the hybrid search requirement.
- **Method:** GET
- **Query Parameters:**
 - query (string): The text search term (e.g., "hp laptop").
 - category (string, optional): Filter by category.
 - max_price (number, optional): Filter for budget.
 - sort (string, optional): relevance (default, uses text search score), price_asc, price_desc, popularity.
- **Example Response (200 OK):**

JSON

```
{
  "search_results": [
    {
      "product_id": "ObjectId(...)",
      "name": "HP Laptop 15",
      "price": 200,000,
      "review_summary": { "average_rating": 4.5, "review_count": 120 }
    },
    // ... other results
  ]
}
```

2. GET /users/<id>/orders

- **Description:** Retrieves the order history for a specific user. (This endpoint must be protected by authentication).
- **Method:** GET
- **URL Parameter:**
 - <id> (ObjectId): The id of the user.
- **Example Response (200 OK):**

JSON

```
{
  "user_id": "ObjectId('user_id_1')",
  "orders": [
    {
      "order_id": "ObjectId('order_id_1')",
      "total_cost": 350,000,
      "status": "shipped",
      "timestamp": "ISODate('...)"
    },
    // ... other orders
  ]
}
```

3. GET /products/<id>/reviews

- **Description:** Retrieves all reviews for a specific product.
- **Method:** GET
- **URL Parameter:**
 - <id> (ObjectId): The id of the product.
- **Example Response (200 OK):**

JSON

```
{
  "product_id": "ObjectId('product_id_1')",
  "reviews": [
    {
      "review_id": "ObjectId('...')",
      "user_name": "Mahad.", // Populated from user_id
      "rating": 5,
      "review_text": "Amazing laptop!",
      "timestamp": "ISODate('...')",
    },
    // ... other reviews
  ]
}
```

4. GET /orders/<id>

- **Description:** Retrieves the full details for a single order. (This endpoint must be protected).
- **Method:** GET
- **URL Parameter:**
 - <id> (ObjectId): The id of the order.
- **Example Response (200 OK):**

JSON

```
{
  "_id": "ObjectId('order_id_1')",
  "user_id": "ObjectId('user_id_1')",
  "products": [
    {
      "product_id": "ObjectId('product_id_1')",
      "name": "HP Envy x360 14",
    }
  ]
}
```

```

        "price_at_purchase": 250,000,
        "quantity": 1
    }
],
    "total_cost": 250,000,
    "status": "shipped",
    "shipping_address": { "address": "123 Main St", ... },
    "timestamp": "ISODate(...)"
}

```

Task 4: Aggregation Pipeline

Query: "Top 5 most frequently purchased products in the last month, grouped by category."

Aggregation Pipeline Code:

```

import pymongo
from datetime import datetime, timedelta

# --- 1. Setup Connection (Example) ---

client = pymongo.MongoClient("mongodb://localhost:27017/")
db = client["ecommerce_db"]
orders_collection = db["orders"]

# --- 2. Calculate Date ---
# Get the datetime object for 30 days ago
thirty_days_ago = datetime.now() - timedelta(days=30)

# --- 3. Define the Aggregation Pipeline ---
pipeline = [
    # Stage 1: Filter orders to the last 30 days
    {
        "$match": {
            "timestamp": {
                "$gte": thirty_days_ago
            }
        }
    },

    # Stage 2: Deconstruct the 'products' array
    {"$unwind": "$products"},

    # Stage 3: Group by product_id and sum quantities purchased
    {
        "$group": {
            "_id": "$products.product_id",
            "total_purchased": {"$sum": "$products.quantity"}
        }
    }
]

```

```

    }
},

# Stage 4: Join with the 'products' collection to get product details
{
    "$lookup": {
        "from": "products",
        "localField": "_id",
        "foreignField": "_id",
        "as": "product_details"
    }
},

# Stage 5: Deconstruct the joined array
{"$unwind": "$product_details"},

# Stage 6: Sort ALL products by total purchased (descending)
{"$sort": {"total_purchased": -1}},

# Stage 7: Group products by their category
{
    "$group": {
        "_id": "$product_details.category",
        "top_products_in_category": {
            "$push": {
                "product_id": "$_id",
                "product_name": "$product_details.name",
                "total_purchased": "$total_purchased"
            }
        }
    }
},

# Stage 8: Slice the array to get only the Top 5 from each category
{
    "$project": {
        "category": "$_id",
        "top_5_products": {"$slice": ["$top_products_in_category", 5]},
        "_id": 0
    }
}
]

# --- 4. Execute the Pipeline ---
try:
    results = list(orders_collection.aggregate(pipeline))

    # Print the results
    for category_data in results:
        print(f"Category: {category_data['category']}")
        for product in category_data['top_5_products']:
            print(f"  - Product: {product['product_name']} (ID: {product['product_id']})")
            print(f"    Purchased: {product['total_purchased']} times")
            print("-" * 20)
except Exception as e:

```

```
print(f"An error occurred: {e}")

finally:
    # Close the client connection
    client.close()
```

The Pipeline Stages:

- **Step 1: Filter Orders by Date** First, the database looks at the entire "orders" collection. It goes through every single order and checks its "timestamp" (the date it was placed). It only keeps the orders that happened in the last 30 days and discards all older ones.
- **Step 2: Unroll the Product Lists** An order can contain multiple products in a list. To count products individually, we need to "unroll" or "flatten" this list. If one order had a laptop and a mouse, this stage splits it into two separate entries: one for the laptop and one for the mouse from that order.
- **Step 3: Count Total Sales for Each Product** Now we have a very long list of every individual product sold in the last month. This stage groups them by their unique product ID. It then "sums" the "quantity" for each group. For example, if "Product A" was sold 10 times (in 5 different orders), its final count will be 10. The result is a new list showing each product's ID and its total sales count.
- **Step 4: Look Up Product Details** At this point, we only have product IDs (like '123-ABC'), but not their names or categories. This stage takes our list of IDs and "looks them up" in the main "products" collection. It finds the matching product document and pulls in its details, like "name," "brand," and most importantly, "category."
- **Step 5: Unpack the Product Details** The "lookup" in the previous step adds the product details as a small, one-item list. This stage simply "unpacks" that list, making the product details (like "category") easier to access for the next steps.
- **Step 6: Sort All Products by Popularity** We now have a complete list of every product sold, its total sales count, and its category. This step sorts this entire, combined list. It places the products with the highest "total purchased" count at the very top and the least popular at the bottom.
- **Step 7: Group Products by Their Category** Now that the list is sorted by popularity, this stage groups all the products based on their "category." It creates new lists: one for "Electronics," one for "Books," one for "Clothing," and so on. As it groups them, it "pushes" the product information (name and sales count) into the correct category's list. Because we sorted before grouping, the most popular products automatically go to the beginning of each category's new list.
- **Step 8: Show Only the Top 5 for Each Category** The lists from Step 7 are complete but show all products. This final stage "slices" each category's list, keeping only the first 5 items (which we know are the most popular because of Step 6). It also cleans up the output, for example, by renaming the grouping "ID" to just "category," giving us the final, clean report.

Q2)

Project Structure:

The project was implemented using the following technologies:

- **Language:** Python 3.12
- **Web Framework:** FastAPI
- **Database:** MongoDB (running locally on `localhost:27017`)
- **Python Driver:** PyMongo
- **Data Validation:** Pydantic v1.10.x
- **Server:** Uvicorn

The final project is organized as follows:

```
ADS_MID/
├── .venv/                <-- Virtual environment
├── dataset/              <-- Sample data files for submission
│   ├── products.json
│   └── README.md
├── main.py               <-- FastAPI application with all API endpoints
├── seed.py               <-- Script to seed the database and create indexes
├── requirements.txt      <-- All Python dependencies
└── README.md             <-- Main project README with setup instructions
```

Implementation Details:

Data Seeding (seed.py)

The database is populated using a Python script. This method was chosen over static JSON files because:

1. **Data Linking:** It allows for the creation of `ObjectIds` as Python variables, which are then used to perfectly link related documents (e.g., a user's `purchase_history` array contains the exact `_ids` of their orders).
2. **Data Types:** It inserts native Python datetime objects, which MongoDB correctly stores as BSON dates.

The script performs four actions:

1. Drops all old collections to ensure a clean state.
2. Inserts the linked data for users, products, orders, and reviews.

3. Creates the **Text Search Index** on the products collection, covering the name, description, brand, and category fields.
4. Creates all other required indexes for query optimization (e.g., on user_id, product_id, etc.).

API Server (main.py)

The API server is built with FastAPI. Key features include:

- **Pydantic Models:** Uses Pydantic v1 to define strict data models (e.g., Order, Review) for request and response validation.
- **Custom PyObjectId Type:** A custom class was created to allow Pydantic to validate and serialize MongoDB's ObjectId.
- **Implemented Endpoints:** The three required APIs (/orders/{id}, /users/{id}/orders, /products/{id}/reviews) and the search API were fully implemented.

Search Implementation:

The GET /products/search endpoint was implemented to satisfy both the keyword and hybrid ranking requirements.

Keyword & Fuzzy Search

- **Keyword Search:** This is handled by **Stage 1** of the aggregation pipeline, which uses the \$text operator to search against the text index we created.
- **Similarity Search:** The text index also provides basic "fuzzy" matching, such as stemming (e.g., "engineering" would match "engineer") and tolerance for minor misspellings.

Hybrid Ranking:

The core logic is a multi-stage aggregation pipeline that calculates a final weighted_score for each product:

Stage 1: \$match

- Filters the entire product list down to only those matching the search query (e.g., "electronics").

Stage 2: \$addFields (Scoring)

- **Similarity (40%):** Calculates a similarity_score using {"\$meta": "textScore"}.

- **Popularity (40%):** Calculates a popularity_score using {"\$log10": ...} on the purchase_count field. This scales the popularity so one viral item doesn't dominate.
- **Price (20%):** Calculates a price_score based on the budget parameter. The formula $1 - (\text{distance_from_budget} / \text{budget})$ is used, (e.g., a \$99 product with a \$100 budget gets a score near 1.0).

Stage 3: \$group & \$unwind (Normalization)

- Finds the max_similarity and max_popularity across all results.
- Divides each product's score by the max to normalize all scores to a 0-1 range. This is essential for combining them fairly.

Stage 4: \$addFields (Final Weighted Score)

- Calculates the final score using the rubric's formula: $(\text{norm_similarity} * 0.4) + (\text{norm_popularity} * 0.4) + (\text{price_score} * 0.2)$

Stage 5: \$sort

- Sorts the final list by the weighted_score in descending order.

Code:

Main.py

```
import pymongo
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field
from bson.objectid import ObjectId
import uvicorn
from typing import List, Optional
from datetime import datetime

# --- Pydantic Models (Data Validation) ---

# --- Pydantic v1 Class for ObjectId ---
class PyObjectId(ObjectId):
    @classmethod
    def __get_validators__(cls):
        yield cls.validate

    @classmethod
    def validate(cls, v):
        if not ObjectId.is_valid(v):
            raise ValueError("Invalid objectid")
        return ObjectId(v)
```

```

    @classmethod
    def __modify_schema__(cls, field_schema):
        field_schema.update(type="string")

# --- Models using Pydantic v1 'class Config' ---

# Model for displaying a single product in an order
class OrderProduct(BaseModel):
    product_id: PyObjectId = Field(..., alias="_id")
    name: str
    price_at_purchase: float
    quantity: int

    class Config:
        allow_population_by_field_name = True
        json_encoders = {ObjectId: str}

# Model for displaying a single order's full details
class Order(BaseModel):
    id: PyObjectId = Field(..., alias="_id")
    user_id: PyObjectId
    products: List[OrderProduct]
    total_cost: float
    status: str
    shipping_address: dict
    timestamp: datetime

    class Config:
        allow_population_by_field_name = True
        json_encoders = {ObjectId: str}

# Model for just the order summary (for the user's list)
class OrderSummary(BaseModel):
    id: PyObjectId = Field(..., alias="_id")
    total_cost: float
    status: str
    timestamp: datetime

    class Config:
        allow_population_by_field_name = True
        json_encoders = {ObjectId: str}

# Model for displaying a single review
class Review(BaseModel):
    id: PyObjectId = Field(..., alias="_id")
    product_id: PyObjectId
    user_id: PyObjectId
    rating: int
    review_text: str
    timestamp: datetime

    class Config:
        allow_population_by_field_name = True

```

```

        json_encoders = {ObjectId: str}

class ProductSearchResult(BaseModel):
    id: PyObjectId = Field(..., alias="_id")
    name: str
    price: float
    category: str
    brand: str
    review_summary: dict

    final_score: float = Field(..., alias="weighted_score")
    similarity_score: float
    popularity_score: float
    price_score: float

    class Config:
        allow_population_by_field_name = True
        json_encoders = {ObjectId: str}

# --- Application Setup ---
app = FastAPI(
    title="E-commerce API",
    description="Backend for the E-commerce Marketplace (Q2)",
    version="1.0.0"
)

# --- Database Connection ---
try:
    client = pymongo.MongoClient("mongodb://localhost:27017/")
    db = client["ecom_marketplace"]
    print("Connected to MongoDB successfully!")
except pymongo.errors.ConnectionFailure as e:
    print(f"Could not connect to MongoDB: {e}")
    db = None

# --- API Endpoints ---

@app.get("/")
def read_root():
    if db is None:
        raise HTTPException(status_code=503, detail="Database connection not available")
    return {"message": "Welcome to the E-commerce API!", "db_status": "connected"}

# --- API 1: Get Order Details ---
@app.get("/orders/{id}", response_model=Order)
def get_order_by_id(id: str):
    if db is None:
        raise HTTPException(status_code=503, detail="Database connection not available")
    try:
        order_id = ObjectId(id)

```

```

    except Exception:
        raise HTTPException(status_code=400, detail="Invalid Order ID
format")
    order = db.orders.find_one({"_id": order_id})
    if order:
        return order
    raise HTTPException(status_code=404, detail=f"Order with id {id} not
found")

# --- API 2: Get User's Orders ---
@app.get("/users/{id}/orders", response_model=List[OrderSummary])
def get_user_orders(id: str):
    if db is None:
        raise HTTPException(status_code=503, detail="Database connection not
available")
    try:
        user_id = ObjectId(id)
    except Exception:
        raise HTTPException(status_code=400, detail="Invalid User ID format")
    user = db.users.find_one({"_id": user_id}, {"purchase_history": 1})
    if not user:
        raise HTTPException(status_code=404, detail=f"User with id {id} not
found")
    order_ids = user.get("purchase_history", [])
    if not order_ids:
        return []
    orders_cursor = db.orders.find(
        {"_id": {"$in": order_ids}}
    ).sort("timestamp", pymongo.DESCENDING)
    return list(orders_cursor)

# --- API 3: Get Product Reviews ---
@app.get("/products/{id}/reviews", response_model=List[Review])
def get_product_reviews(id: str):
    if db is None:
        raise HTTPException(status_code=503, detail="Database connection not
available")
    try:
        product_id = ObjectId(id)
    except Exception:
        raise HTTPException(status_code=400, detail="Invalid Product ID
format")
    product = db.products.find_one({"_id": product_id}, {"_id": 1})
    if not product:
        raise HTTPException(status_code=404, detail=f"Product with id {id}
not found")
    reviews_cursor = db.reviews.find(
        {"product_id": product_id}
    ).sort("timestamp", pymongo.DESCENDING)
    return list(reviews_cursor)

# --- API 4: Search ---
@app.get("/products/search", response_model=List[ProductSearchResult])
def search_products(

```

```

        query: str,
        budget: Optional[float] = None,
        limit: int = 10
    ):
        if db is None:
            raise HTTPException(status_code=503, detail="Database connection not
available")

        pipeline = []

        # --- Stage 1: Text Search (Keyword Search) ---
        pipeline.append({
            "$match": {
                "$text": {"$search": query}
            }
        })

        # --- Stage 2: Calculate Initial Scores (Hybrid Ranking) ---
        pipeline.append({
            "$addFields": {
                "similarity_score": {"$meta": "textScore"},
                "popularity_score": {"$log10": {"$add": [{"$purchase_count", 1}]}},
                "price_score": {
                    "$cond": {
                        "if": {"$and": [budget, {"$gt": [budget, 0]}]},
                        "then": {
                            # --- THIS IS THE NEW, FIXED CODE ---
                            "$max": [
                                0,
                                {
                                    "$subtract": [
                                        1,
                                        {
                                            "$abs": {
                                                "$divide": [
                                                    { "$subtract": ["$price", budget] },
                                                    budget
                                                ]
                                            }
                                        ]
                                    }
                                ]
                            }
                        },
                        "else": 0
                    }
                }
            }
        })

        # --- END OF FIXED CODE ---

        # --- Stage 3: Normalize Scores ---
        pipeline.append({
            "$group": {
                "_id": None,
                "products": {"$push": "$$ROOT"},
                "max_similarity": {"$max": "$similarity_score"},
            }
        })

```

```

        "max_popularity": {"$max": "$popularity_score"} # <-- Bug fix
from earlier is included
    }
  })

  pipeline.append({"$unwind": "$products"})

  pipeline.append({
    "$addFields": {
      "norm_similarity": {
        "$cond": {
          "if": {"$gt": ["$max_similarity", 0]},
          "then": {"$divide": ["$products.similarity_score",
"$max_similarity"]},
          "else": 0
        }
      },
      "norm_popularity": {
        "$cond": {
          "if": {"$gt": ["$max_popularity", 0]},
          "then": {"$divide": ["$products.popularity_score",
"$max_popularity"]},
          "else": 0
        }
      },
      "price_score": "$products.price_score"
    }
  })

  # --- Stage 4: Calculate Final Weighted Score (The Bonus) ---
  pipeline.append({
    "$addFields": {
      "weighted_score": {
        "$add": [
          {"$multiply": ["$norm_similarity", 0.4]},
          {"$multiply": ["$norm_popularity", 0.4]},
          {"$multiply": ["$price_score", 0.2]}
        ]
      },
      "name": "$products.name",
      "_id": "$products._id",
      "price": "$products.price",
      "category": "$products.category",
      "brand": "$products.brand",
      "review_summary": "$products.review_summary",
      "similarity_score": "$products.similarity_score",
      "popularity_score": "$products.popularity_score"
    }
  })

  # --- Stage 5: Sort by Final Score and Limit ---
  pipeline.append({"$sort": {"weighted_score": -1}})
  pipeline.append({"$limit": limit})

  # --- Stage 6: Clean up final output ---
  pipeline.append({
    "$project": {

```

```

        "_id": 1,
        "name": 1,
        "price": 1,
        "category": 1,
        "brand": 1,
        "review_summary": 1,
        "weighted_score": 1,
        "similarity_score": "$norm_similarity",
        "popularity_score": "$norm_popularity",
        "price_score": 1
    }
})

# --- Execute the Pipeline ---
try:
    results = list(db.products.aggregate(pipeline))
    return results
except Exception as e:
    print(f"Error during search aggregation: {e}")
    raise HTTPException(status_code=500, detail="Search query failed")

# --- Run the App ---
if __name__ == "__main__":
    print("Starting FastAPI server at http://127.0.0.1:8000")
    uvicorn.run(
        "main:app",
        host="127.0.0.1",
        port=8000,
        reload=True
    )

```

seed.py

```

import pymongo
from datetime import datetime
from bson.objectid import ObjectId
import sys

# --- Configuration ---
MONGO_URI = "mongodb://localhost:27017/"
DB_NAME = "ecom_marketplace"

# --- Data Definitions ---

# We'll pre-define ObjectIDs so we can link documents
user_sufyan_id = ObjectId()
user_jane_id = ObjectId()

prod_laptop_id = ObjectId()
prod_mouse_id = ObjectId()
prod_book_id = ObjectId()

```

```

def get_data():
    """Defines the expanded seed data."""

    # --- ObjectIDs for linking ---
    # Users
    user_sufyan_id = ObjectId()
    user_jane_id = ObjectId()
    user_ali_id = ObjectId()

    # Products
    prod_laptop_hp_id = ObjectId()
    prod_mouse_logi_id = ObjectId()
    prod_book_pragmatic_id = ObjectId()
    prod_keyboard_keychron_id = ObjectId()
    prod_monitor_dell_id = ObjectId()
    prod_book_design_id = ObjectId()
    prod_coffee_maker_id = ObjectId()
    prod_headphones_sony_id = ObjectId()

    # Orders
    order_1_id = ObjectId()
    order_2_id = ObjectId()
    order_3_id = ObjectId()
    order_4_id = ObjectId()

    # --- Collections ---

    users = [
        {
            "_id": user_sufyan_id,
            "name": "Sufyan Ahmed",
            "email": "sufyan@example.com",
            "hashed_password": "bcrypt_hash_string_1",
            "location": {"address": "123 Main St", "city": "Lahore"},
            "country": "Pakistan",
            "purchase_history": [order_1_id], # Will be updated
            "created_at": datetime(2025, 1, 15)
        },
        {
            "_id": user_jane_id,
            "name": "Jane Doe",
            "email": "jane@example.com",
            "hashed_password": "bcrypt_hash_string_2",
            "location": {"address": "456 Oak Ave", "city": "New York"},
            "country": "USA",
            "purchase_history": [order_2_id, order_4_id],
            "created_at": datetime(2025, 2, 20)
        },
        {
            "_id": user_ali_id,
            "name": "Ali Raza",
            "email": "ali@example.com",
            "hashed_password": "bcrypt_hash_string_3",
            "location": {"address": "789 Pine Ln", "city": "Karachi"},
            "country": "Pakistan",
            "purchase_history": [order_3_id],
            "created_at": datetime(2025, 5, 10)
        }
    ]

```

```

    ]

    products = [
        {
            "_id": prod_laptop_hp_id,
            "name": "HP Spectre x360 14",
            "description": "Premium 2-in-1 laptop with 14-inch display.",
            "category": "Electronics",
            "brand": "HP",
            "price": 1499.99,
            "stock": 150,
            "specs": {"ram_gb": 16, "storage_ssd_gb": 512, "processor":
"Intel Core i7"},
            "review_summary": {"average_rating": 4.7, "review_count": 2}, #
Updated count
            "purchase_count": 120,
            "created_at": datetime(2025, 3, 1)
        },
        {
            "_id": prod_mouse_logi_id,
            "name": "Logitech MX Master 3",
            "description": "Advanced wireless mouse for productivity.",
            "category": "Electronics",
            "brand": "Logitech",
            "price": 99.99,
            "stock": 300,
            "specs": {"connectivity": "Bluetooth/USB", "dpi": 4000},
            "review_summary": {"average_rating": 4.8, "review_count": 1}, #
Updated count
            "purchase_count": 450,
            "created_at": datetime(2025, 3, 5)
        },
        {
            "_id": prod_book_pragmatic_id,
            "name": "The Pragmatic Programmer",
            "description": "A classic book on software engineering and
coding.",
            "category": "Books",
            "brand": "Addison-Wesley",
            "price": 45.50,
            "stock": 100,
            "specs": {"format": "Paperback", "pages": 352},
            "review_summary": {"average_rating": 4.6, "review_count": 1}, #
Updated count
            "purchase_count": 75,
            "created_at": datetime(2025, 1, 10)
        },
        {
            "_id": prod_keyboard_keychron_id,
            "name": "Keychron Q2 Mechanical Keyboard",
            "description": "A fully customizable 65% layout mechanical
keyboard.",
            "category": "Electronics",
            "brand": "Keychron",
            "price": 189.99,
            "stock": 75,

```

```

        "specs": {"layout": "65%", "switch": "Gateron Brown"},
        "review_summary": {"average_rating": 4.9, "review_count": 1}, #
Updated count
        "purchase_count": 210,
        "created_at": datetime(2025, 4, 15)
    },
    {
        "_id": prod_monitor_dell_id,
        "name": "Dell UltraSharp 27\" 4K Monitor",
        "description": "A 4K UHD monitor with stunning color accuracy.",
        "category": "Electronics",
        "brand": "Dell",
        "price": 549.99,
        "stock": 50,
        "specs": {"resolution": "3840x2160", "panel_type": "IPS"},
        "review_summary": {"average_rating": 4.5, "review_count": 0},
        "purchase_count": 35,
        "created_at": datetime(2025, 2, 28)
    },
    {
        "_id": prod_book_design_id,
        "name": "Designing Data-Intensive Applications",
        "description": "The big book on data systems design.",
        "category": "Books",
        "brand": "O'Reilly",
        "price": 59.99,
        "stock": 120,
        "specs": {"format": "Paperback", "pages": 616},
        "review_summary": {"average_rating": 4.9, "review_count": 1}, #
Updated count
        "purchase_count": 180,
        "created_at": datetime(2025, 1, 5)
    },
    {
        "_id": prod_coffee_maker_id,
        "name": "Breville Barista Express",
        "description": "All-in-one espresso machine with integrated
grinder.",
        "category": "Home Appliances",
        "brand": "Breville",
        "price": 749.95,
        "stock": 30,
        "specs": {"type": "Espresso Machine", "water_tank_l": 2},
        "review_summary": {"average_rating": 4.7, "review_count": 0},
        "purchase_count": 40,
        "created_at": datetime(2025, 3, 20)
    },
    {
        "_id": prod_headphones_sony_id,
        "name": "Sony WH-1000XM5",
        "description": "Industry-leading noise-canceling wireless
headphones.",
        "category": "Electronics",
        "brand": "Sony",
        "price": 399.99,
        "stock": 90,
        "specs": {"connectivity": "Bluetooth", "noise_canceling":

```

```

"Active"},
    "review_summary": {"average_rating": 0, "review_count": 0},
    "purchase_count": 0, # New product
    "created_at": datetime(2025, 10, 20) # Very recent
}
]

orders = [
    {
        "_id": order_1_id,
        "user_id": user_sufyan_id,
        "products": [
            {
                "product_id": prod_laptop_hp_id,
                "name": "HP Spectre x360 14",
                "price_at_purchase": 1499.99,
                "quantity": 1
            },
            {
                "product_id": prod_mouse_logi_id,
                "name": "Logitech MX Master 3",
                "price_at_purchase": 99.99,
                "quantity": 1
            }
        ],
        "total_cost": 1599.98,
        "status": "shipped",
        "shipping_address": {"address": "123 Main St", "city": "Lahore",
"country": "Pakistan"},
        "timestamp": datetime(2025, 10, 15) # Last month
    },
    {
        "_id": order_2_id,
        "user_id": user_jane_id,
        "products": [
            {
                "product_id": prod_book_pragmatic_id,
                "name": "The Pragmatic Programmer",
                "price_at_purchase": 45.50,
                "quantity": 2
            }
        ],
        "total_cost": 91.00,
        "status": "delivered",
        "shipping_address": {"address": "456 Oak Ave", "city": "New
York", "country": "USA"},
        "timestamp": datetime(2025, 9, 10)
    },
    {
        "_id": order_3_id,
        "user_id": user_ali_id,
        "products": [
            {
                "product_id": prod_keyboard_keychron_id,
                "name": "Keychron Q2 Mechanical Keyboard",
                "price_at_purchase": 189.99,
                "quantity": 1
            }
        ]
    }
]

```

```

        },
        {
            "product_id": prod_monitor_dell_id,
            "name": "Dell UltraSharp 27\" 4K Monitor",
            "price_at_purchase": 549.99,
            "quantity": 1
        }
    ],
    "total_cost": 739.98,
    "status": "pending",
    "shipping_address": {"address": "789 Pine Ln", "city": "Karachi",
"country": "Pakistan"},
    "timestamp": datetime(2025, 10, 21) # Very recent
},
{
    "_id": order_4_id,
    "user_id": user_jane_id,
    "products": [
        {
            "product_id": prod_book_design_id,
            "name": "Designing Data-Intensive Applications",
            "price_at_purchase": 59.99,
            "quantity": 1
        },
        {
            "product_id": prod_coffee_maker_id,
            "name": "Breville Barista Express",
            "price_at_purchase": 749.95,
            "quantity": 1
        }
    ],
    "total_cost": 809.94,
    "status": "delivered",
    "shipping_address": {"address": "456 Oak Ave", "city": "New
York", "country": "USA"},
    "timestamp": datetime(2025, 8, 5)
}
]

reviews = [
    # Reviews for HP Laptop
    {
        "_id": ObjectId(),
        "product_id": prod_laptop_hp_id,
        "user_id": user_sufyan_id,
        "rating": 5,
        "review_text": "Amazing laptop, battery life is fantastic!",
        "timestamp": datetime(2025, 10, 20)
    },
    {
        "_id": ObjectId(),
        "product_id": prod_laptop_hp_id,
        "user_id": user_jane_id,
        "rating": 4,
        "review_text": "A bit expensive, but build quality is top-
notch.",
        "timestamp": datetime(2025, 10, 21)
    }
]

```

```

    },
    # Review for Mouse
    {
        "_id": ObjectId(),
        "product_id": prod_mouse_logi_id,
        "user_id": user_sufyan_id,
        "rating": 5,
        "review_text": "Best mouse I have ever used.",
        "timestamp": datetime(2025, 10, 20)
    },
    # Review for Pragmatic Programmer book
    {
        "_id": ObjectId(),
        "product_id": prod_book_pragmatic_id,
        "user_id": user_jane_id,
        "rating": 5,
        "review_text": "A must-read for every developer.",
        "timestamp": datetime(2025, 9, 15)
    },
    # Review for Keychron Keyboard
    {
        "_id": ObjectId(),
        "product_id": prod_keyboard_keychron_id,
        "user_id": user_ali_id,
        "rating": 5,
        "review_text": "Love the typing feel. The build is solid
aluminum.",
        "timestamp": datetime(2025, 10, 22)
    },
    # Review for Data-Intensive book
    {
        "_id": ObjectId(),
        "product_id": prod_book_design_id,
        "user_id": user_jane_id,
        "rating": 5,
        "review_text": "The bible of system design. Incredibly dense but
rewarding.",
        "timestamp": datetime(2025, 8, 10)
    }
]

return users, products, orders, reviews

def create_indexes(db):
    """Creates the indexes we designed in Q1."""

    # 1. On users
    db.users.create_index("email", unique=True)

    # 2. On products (CRITICAL for search)
    db.products.create_index(
        [
            ("name", pymongo.TEXT),
            ("description", pymongo.TEXT),
            ("brand", pymongo.TEXT),
            ("category", pymongo.TEXT) # <-- ADDED THIS LINE

```

```

        ],
        name="product_text_search_index"
    )

    # --- THIS IS THE CORRECTED LINE ---
    db.products.create_index([("purchase_count", pymongo.DESCENDING)])

    db.products.create_index([("category", 1), ("price", 1)])

    # 3. On reviews
    db.reviews.create_index([("product_id", 1), ("timestamp", -1)])

    # 4. On orders
    db.orders.create_index([("user_id", 1), ("timestamp", -1)])

    print("Indexes created successfully.")

def main():
    """Main seeding function."""
    try:
        client = pymongo.MongoClient(MONGO_URI)
        db = client[DB_NAME]

        print(f"Connected to MongoDB. Seeding database: {DB_NAME}")

        # --- 1. Get Data ---
        users, products, orders, reviews = get_data()

        # --- 2. Drop Old Collections ---
        print("Dropping old collections...")
        db.users.drop()
        db.products.drop()
        db.orders.drop()
        db.reviews.drop()

        # --- 3. Insert New Data ---
        print("Inserting new data...")
        db.users.insert_many(users)
        db.products.insert_many(products)
        db.orders.insert_many(orders)
        db.reviews.insert_many(reviews)
        print("Data inserted successfully.")

        # --- 4. Create Indexes ---
        print("Creating indexes...")
        create_indexes(db)

        print("--- Seeding Complete ---")

    except pymongo.errors.ConnectionFailure as e:
        print(f"Could not connect to MongoDB: {e}")
        sys.exit(1)
    except Exception as e:
        print(f"An error occurred: {e}")
    finally:
        client.close()

```

```
if __name__ == "__main__":
    main()
```

Screenshots:

E-commerce API1.0.0OAS 3.1

/openapi.json

Backend for the E-commerce Marketplace (Q2)

default

GET / Read Root

GET /orders/{id} Get Order By Id

GET /users/{id}/orders Get User Orders

GET /products/{id}/reviews Get Product Reviews

GET /products/search Search Products

Schemas

HTTPValidationError > Expand all object

Order > Expand all object

OrderProduct > Expand all object

OrderSummary > Expand all object

ProductSearchResult > Expand all object

Review > Expand all object

ValidationError > Expand all object

GET

/products/{id}/reviews

Get Product Reviews

^

Parameters

Cancel

Name	Description
id * required string (path)	68f9114992a0bd9e7d7ca56a

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/products/68f9114992a0bd9e7d7ca56a/reviews' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/products/68f9114992a0bd9e7d7ca56a/reviews
```

Server response

Code

Details

200

Response body

```
[
  {
    "_id": "68f9114992a0bd9e7d7ca572",
    "product_id": "68f9114992a0bd9e7d7ca56a",
    "user_id": "68f9114992a0bd9e7d7ca569",
    "rating": 4,
    "review_text": "A bit expensive, but build quality is top-notch.",
    "timestamp": "2025-10-21T00:00:00"
  },
  {
    "_id": "68f9114992a0bd9e7d7ca571",
    "product_id": "68f9114992a0bd9e7d7ca56a",
    "user_id": "68f9114992a0bd9e7d7ca568",
    "rating": 5,
    "review_text": "Amazing laptop, battery life is fantastic!",
    "timestamp": "2025-10-20T00:00:00"
  }
]
```

Download

Response headers

```
content-length: 439
content-type: application/json
date: Wed, 22 Oct 2025 18:03:08 GMT
server: uvicorn
```

GET

/orders/{id} Get Order By Id

Parameters

Cancel

Name	Description
id * required	
string	68f920d58032b290820c66f5
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/orders/68f920d58032b290820c66f5' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/orders/68f920d58032b290820c66f5
```

Server response

Code	Details
200	Response body <pre>{ "_id": "68f920d58032b290820c66f5", "user_id": "68f920d58032b290820c66ea", "products": [{ "_id": "68f920d58032b290820c66ed", "name": "HP Spectre x360 14", "price_at_purchase": 1499.99, "quantity": 1 }, { "_id": "68f920d58032b290820c66ee", "name": "Logitech MX Master 3", "price_at_purchase": 99.99, "quantity": 1 }], "total_cost": 1599.98, "status": "shipped", "shipping_address": { "address": "123 Main St", "city": "Lahore", "country": "Pakistan" }, "timestamp": "2025-10-15T00:00:00" }</pre> Download

Response headers

```
content-length: 448
content-type: application/json
date: Wed, 22 Oct 2025 18:36:05 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

GET

/products/search

Search Products

⌵

Parameters

Cancel

Name	Description
query * required string (query)	electronics
budget number (query)	100
limit integer (query)	10

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/products/search?query=electronics&budget=100&limit=10' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/products/search?query=electronics&budget=100&limit=10
```

Server response

Code	Details
200	Response body <pre>{ "id": "68f920d58032b290820c66ee", "name": "Logitech MX Master 3", "price": 99.99, "category": "Electronics", "brand": "Logitech", "review_summary": { "average_rating": 4.8, "review_count": 1 }, "weighted_score": 0.9998000000000001, "similarity_score": 1, "popularity_score": 1, "price_score": 0.9998999999999999 }, { "id": "68f920d58032b290820c66f0", "name": "Keychron Q2 Mechanical Keyboard", "price": 109.99, "category": "Electronics", "brand": "Keychron", "review_summary": { "average_rating": 4.9, "review_count": 1 }, "weighted_score": 0.7703030227944292, "similarity_score": 1, "popularity_score": 0.8757075569860732, "price_score": 0.9999999999999999 }</pre> Download Response headers <pre>content-length: 1466 content-type: application/json date: Wed, 22 Oct 2025 18:38:00 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

```
HTTPValidationError ^ Collapse all object
  detail ^ Collapse all array<object>
    Items ^ Collapse all object
      loc ^ Collapse all array<(string | integer)>
        Items ^ Collapse all (string | integer)
          Any of ^ Collapse all (string | integer)
            #0 string
            #1 integer
      msg^ string
      type^ string
```

```
Order ^ Collapse all object
  _id^ string
  user_id^ string
  products^ ^ Collapse all array<object>
    Items ^ Collapse all object
      _id^ string
      name^ string
      price_at_purchase^ number
      quantity^ integer
  total_cost^ number
  status^ string
  shipping_address^ object
  timestamp^ string date-time
```

```
OrderProduct ^ Collapse all object
  _id^ string
  name^ string
  price_at_purchase^ number
  quantity^ integer
```

```
OrderSummary ^ Collapse all object
  _id^ string
  total_cost^ number
  status^ string
  timestamp^ string date-time
```

```
ProductSearchResult ^ Collapse all object
  _id^ string
  name^ string
  price^ number
  category^ string
  brand^ string
  review_summary^ object
  weighted_score^ number
  similarity_score^ number
  popularity_score^ number
  price_score^ number
```

```
Review ^ Collapse all object
  _id^ string
  product_id^ string
  user_id^ string
  rating^ integer
  review_text^ string
  timestamp^ string date-time
```

```
ValidationError ^ Collapse all object
  loc ^ Collapse all array<(string | integer)>
    Items ^ Collapse all (string | integer)
      Any of ^ Collapse all (string | integer)
        #0 string
        #1 integer
  msg^ string
  type^ string
```